

Java Quiz



Java Step 2 Quiz

OO Basics · Objects · Arrays · Classes · Packages · Strings



The wheel picks the student — **one student, one question!**

wheelofnames.com

What is a key weakness of procedural languages (like C, COBOL, BASIC) that Object Orientation addresses?

- 1 They cannot perform mathematical calculations
- 2 Changes to code can easily break the entire application because you can't be sure which code changes which variables
- 3 They cannot be compiled
- 4 They require more memory than object-oriented languages



What is a key weakness of procedural languages (like C, COBOL, BASIC) that Object Orientation addresses?

1

They cannot perform mathematical calculations

2

Changes to code can easily break the entire application because you can't be sure which code changes which variables



3

They cannot be compiled

4

They require more memory than object-oriented languages

✓ Answer 2 — In procedural code, any part of the program can freely change any variable, so a small change can ripple out and break things unpredictably — OO's encapsulation contains that risk



What is the relationship between a class and an object ?

1 A class is a template/definition for a type of object; an object is an instance of a class, created via instantiation

2 A class and an object are the same thing

3 An object is a template used to create classes

4 A class can only ever be used to create exactly one object



What is the relationship between a `class` and an `object` ?

1 A class is a template/definition for a type of object; an object is an instance of a class, created via instantiation



2 A class and an object are the same thing

3 An object is a template used to create classes

4 A class can only ever be used to create exactly one object

✓ Answer 1 — A class is not an object itself — it's the blueprint. You instantiate a class to create as many (or as few) objects of that type as you need



Of the three "golden rules" of OO, what does Polymorphism mean?

- 1 That every class must implement every method
- 2 That a class can inherit all the features of another class
- 3 That the declared type of a variable and its actual run-time type can be different
- 4 That properties must always be private



Question 3

Java

Of the three "golden rules" of OO, what does Polymorphism mean?

- 1 That every class must implement every method
- 2 That a class can inherit all the features of another class
- 3 That the declared type of a variable and its actual run-time type can be different ✓
- 4 That properties must always be private

✓ Answer 3 — Polymorphism lets you loop through an array declared as Account that actually holds a mix of subclass objects like OffShoreAccount — the correct overridden method is chosen at runtime based on the real object type



Question 4

Java

In `Account acc = new Account();`, what does the variable `acc` actually store?

1 The variable stores the object directly, byte for byte

2 The variable stores a reference (memory address) that points to the object, not the object itself

3 The variable stores a copy of the class definition

4 The variable stores the object's private fields as text



Question 4

Java

In `Account acc = new Account();`, what does the variable `acc` actually store?

1

The variable stores the object directly, byte for byte

2

The variable stores a reference (memory address) that points to the object, not the object itself



3

The variable stores a copy of the class definition

4

The variable stores the object's private fields as text



Answer 2 — `Account acc = new Account();` makes `acc` a reference — a memory address — pointing to the real object elsewhere on the heap



What happens to an object once no reference variable points to it anymore?

- 1 It is instantly deleted from memory
- 2 It becomes corrupted and causes a runtime error
- 3 It becomes eligible for garbage collection, and its memory will eventually be reclaimed
- 4 It automatically converts itself to null



What happens to an object once no reference variable points to it anymore?

1 It is instantly deleted from memory

2 It becomes corrupted and causes a runtime error

3 It becomes eligible for garbage collection, and its memory will eventually be reclaimed



4 It automatically converts itself to null

✓ **Answer 3** — Once nothing references an object, it's earmarked for garbage collection — the JVM reclaims that memory when it needs to (low memory, idle time, or a `System.gc()` hint)



When an object is created, what are the default initial values of its instance variables?

- 1 Primitives default to 0, 0.0, or false; object references default to null
- 2 Every instance variable defaults to null, regardless of type
- 3 Every instance variable must be manually initialised or the code won't compile
- 4 Primitives default to 1 and object references default to an empty object



When an object is created, what are the default initial values of its instance variables?

1 Primitives default to 0, 0.0, or false; object references default to null



2 Every instance variable defaults to null, regardless of type

3 Every instance variable must be manually initialised or the code won't compile

4 Primitives default to 1 and object references default to an empty object

✓ Answer 1 — When an object is created with new, every instance variable that isn't explicitly initialised gets a default value automatically — 0/0.0/false for primitives, null for object references



Question 7

Java

After `Account[] myAccounts = new Account[5];`, **what are the array's contents?**

1 Five empty `Account` objects, ready to use

2 A compile error, because arrays cannot hold object references

3 Five copies of the same default `Account` object

4 Five references, all pointing to null — you still need to instantiate each element separately



Question 7

Java

After `Account[] myAccounts = new Account[5];`, **what are the array's contents?**

1 Five empty Account objects, ready to use

2 A compile error, because arrays cannot hold object references

3 Five copies of the same default Account object

4 Five references, all pointing to null — you still need to instantiate each element separately



✓ Answer 4 — Creating an array of object references only reserves five slots for references — each one starts out null until you explicitly do `myAccounts[i] = new Account();`



What's the difference between a traditional `for` loop and a `for-each` loop over an array?

1 `for-each` is not available for arrays, only for collections

2 A traditional `for` loop is always faster and should always be preferred

3 A `for-each` loop like `for (Account a : myAccounts)` gives you each element directly without managing an index, while the traditional `for` loop needs an index and array length

4 `for-each` can only be used with arrays of primitives



What's the difference between a traditional `for` loop and a `for-each` loop over an array?

1 `for-each` is not available for arrays, only for collections

2 A traditional `for` loop is always faster and should always be preferred

3 A `for-each` loop like `for (Account a : myAccounts)` gives you each element directly without managing an index, while the traditional `for` loop needs an index and array length ✓

4 `for-each` can only be used with arrays of primitives

✓ Answer 3 — The Java 5 `for-each` loop hides the index bookkeeping and just hands you each element in turn — useful when you don't need the index itself



What happens to the compiler-provided default no-argument constructor once you define your own constructor in a class?

1 The default constructor is no longer present; you'd need to explicitly add a no-arg constructor yourself if you still want one

2 Nothing changes — the default constructor is still available alongside yours

3 The class becomes abstract automatically

4 Java throws a compile-time error demanding you delete your custom constructor



What happens to the compiler-provided default no-argument constructor once you define your own constructor in a class?

1

The default constructor is no longer present; you'd need to explicitly add a no-arg constructor yourself if you still want one



2

Nothing changes — the default constructor is still available alongside yours

3

The class becomes abstract automatically

4

Java throws a compile-time error demanding you delete your custom constructor

 **Answer 1** — The compiler only supplies the free no-arg constructor if you define no constructor at all — as soon as you add any constructor, that freebie disappears



What is the purpose of the `this` keyword inside `setBalance(double balance) { this.balance = balance; }`?

1 It creates a new object automatically

2 It calls the parent class's constructor

3 It marks the method as static

4 It refers to the current object, disambiguating the instance variable from a parameter/local variable that has the same name



Question 10

Java

What is the purpose of the `this` keyword inside `setBalance(double balance) { this.balance = balance; }`?

1 It creates a new object automatically

2 It calls the parent class's constructor

3 It marks the method as static

4 It refers to the current object, disambiguating the instance variable from a parameter/local variable that has the same name



✓ Answer 4 — When a parameter and an instance variable share a name, `this.balance` clearly means the instance variable belonging to the current object, versus the plain `balance` which refers to the parameter



What is method overloading ?

1 Writing a method that never returns a value

2 Having multiple methods in the same class share the same name but take different parameter lists, like `withdraw()` and `withdraw(double d)`

3 Calling a method more times than it was designed for

4 Making a method static so it can be called without an object



What is method overloading ?

1 Writing a method that never returns a value

2 Having multiple methods in the same class share the same name but take different parameter lists, like `withdraw()` and `withdraw(double d)` 

3 Calling a method more times than it was designed for

4 Making a method static so it can be called without an object

 Answer 2 — Overloaded methods share a name but are distinguished by their parameter list — the compiler picks the right one based on the arguments you pass



Why are Java classes organised into packages ?

- 1 To make the compiler run faster
- 2 Packages are required only for classes that use inheritance
- 3 To group related classes by functionality and avoid naming clashes between classes
- 4 To automatically generate getters and setters



Why are Java classes organised into packages ?

- 1 To make the compiler run faster
- 2 Packages are required only for classes that use inheritance
- 3 To group related classes by functionality and avoid naming clashes between classes ✓
- 4 To automatically generate getters and setters

✓ Answer 3 — Packages let the whole Java library (and your own code) be organised by functionality, and two classes with the same simple name can coexist as long as they're in different packages



In enterprise-scale Java projects, how are external library dependencies typically managed, rather than manually adding jars to an IDE's build path?

1 Using a build tool like Maven or Gradle to manage dependencies consistently across the team and enable automated builds

2 By emailing jar files between developers

3 By copying every dependency's source code directly into your own classes

4 Dependencies aren't needed in enterprise Java projects



Question 13

Java

In enterprise-scale Java projects, how are external library dependencies typically managed, rather than manually adding jars to an IDE's build path?

1 Using a build tool like Maven or Gradle to manage dependencies consistently across the team and enable automated builds ✓

2 By emailing jar files between developers

3 By copying every dependency's source code directly into your own classes

4 Dependencies aren't needed in enterprise Java projects

✓ **Answer 1 — Manually adding jars to an IDE's build path is hard to keep consistent across multiple developers and breaks automated builds — Maven/Gradle solve this by declaring dependencies in a build file**



Question 14

Java

Given `String name1 = "hello";` **and** `String name3 = new String("hello");` , **why should you compare them with** `.equals()` **instead of** `==` ?

1 `==` only works on primitive types, never on any object

2 `.equals()` is faster than `==` for every type of comparison

3 `==` always throws a compile error when used on Strings

4 `name1` comes from the shared String pool while `new String(...)` always creates a brand-new object in memory, so `==` would return false even though the text is identical — `.equals()` compares the actual content



Question 14

Java

Given `String name1 = "hello";` **and** `String name3 = new String("hello");` , **why should you compare them with** `.equals()` **instead of** `==` ?

1 `==` only works on primitive types, never on any object

2 `.equals()` is faster than `==` for every type of comparison

3 `==` always throws a compile error when used on Strings

4 `name1` comes from the shared String pool while `new String(...)` always creates a brand-new object in memory, so `==` would return false even though the text is identical — `.equals()` compares the actual content 



Why might `StringBuilder` be preferred over `String` for text that changes frequently?

1 `StringBuilder` objects are immutable, just like `String`

2 Every `String` manipulation method returns a brand-new `String` object (since `Strings` are immutable), which has performance costs for frequent changes; `StringBuilder` can be modified in place

3 `StringBuilder` can only hold numbers, not text

4 `String` is always faster than `StringBuilder`, but less safe



Why might `StringBuilder` be preferred over `String` for text that changes frequently?

1 `StringBuilder` objects are immutable, just like `String`

2 Every `String` manipulation method returns a brand-new `String` object (since `Strings` are immutable), which has performance costs for frequent changes; `StringBuilder` can be modified in place 

3 `StringBuilder` can only hold numbers, not text

4 `String` is always faster than `StringBuilder`, but less safe

 Answer 2 — Because `String` is immutable, methods like `concat()` or `replace()` must allocate a new `String` each time — `StringBuilder` instead mutates an internal buffer, which is much cheaper for text that changes a lot





Well Done!

That's the end of the Java Step 2 quiz.

OO fundamentals, classes & objects, references & garbage collection, arrays, constructors & this, overloading, static members, packages & dependencies, String pool & equality, StringBuilder: ✓